Wardrobe Agent

Fiche technique : ce qu'il fait, ce qu'il utilise, ses prompts, ses paramètres et ses fichiers

Le Wardrobe Agent est **le propriétaire de la garde-robe** de l'utilisateur. Il cherche, compte, compare et corrige des vêtements. C'est aussi le seul point par lequel les autres agents lisent la garde-robe.

Carte d'identité

Question	Réponse
Utilise-t-il un LLM ?	**Oui, mais pas partout.** Texte libre, ask et check-duplicate : oui. list, search, calculs, validation, écriture : non (voir la partie 2).
Quel LLM ?	OpenAI **gpt-4o** (défaut de `LLM_MODEL`), appelé par l'API Chat Completions avec appel d'outils
Température	**0,2** pour répondre (`node_assist`), **0,1** pour la détection de doublon
Technique	graphe LangGraph à 4 nœuds + boucle d'outils partagée `agent_kit/tool_loop.py`
Outils	4 : `search_wardrobe`, `find_similar`, `wardrobe_stats`, `update_item`
Peut-il écrire ?	**Non, jamais seul.** Il propose ; le code écrit après un « oui » de l'utilisateur
Où ?	BO1, port 8000, fichier `backend/app/wardrobe_agent.py` (491 lignes)
Interfaces	`POST /api/wardrobe/agent`, A2A `/a2a/wardrobe`, `POST /api/agent/check-duplicate`
Mémoire	aucune conversation longue ; seulement les propositions de correction en attente (15 minutes)

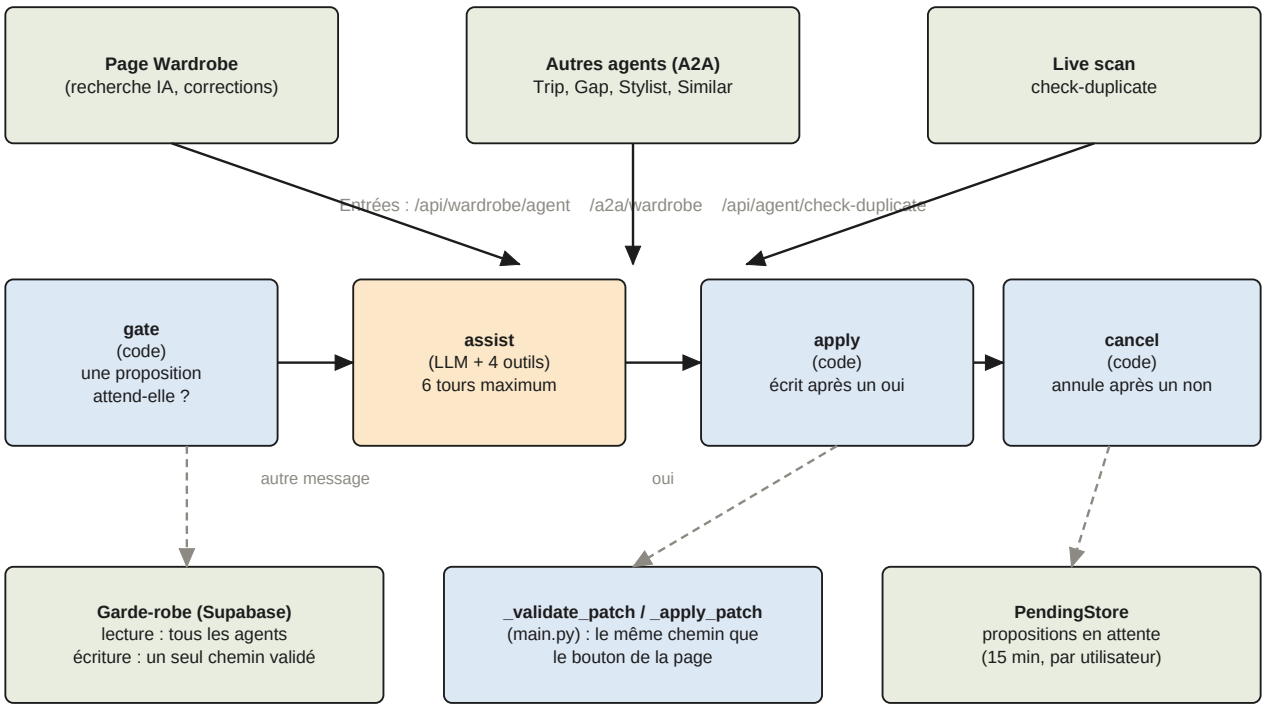

Figure 1 - les entrées, le graphe et ce qu'il touche.

1. Les tâches qu'il fait

Tâche	Exemple	Comment
Chercher	« Montre-moi une veste chaude pour le travail »	le LLM choisit les filtres, le code filtre (search_wardrobe)
Compter et résumer	« Combien de hauts ai-je ? »	wardrobe_stats : calculé par le code
Comparer	« Ai-je déjà quelque chose comme mon blazer marine ? »	find_similar : score calculé par le code
Corriger une pièce	« Change la couleur de ma jupe grise en bleu marine »	update_item **propose** ; le code écrit après un oui
Détecter un doublon	pièce qu'on vient de scanner	agent à part (prompt dédié), 4 tours, JSON strict
Servir les autres agents	Trip Agent : toute la garde-robe ; AI Stylist : « une veste pour le travail »	list, search, ask via A2A

Ce qu'il ne fait **pas** : conseiller une tenue, consulter la météo ou les livres de mode, construire des tenues. Son prompt lui interdit et le renvoie à l'AI Stylist et à l'Outfit Builder.

2. Où il utilise un LLM, et où non

Opération	LLM ?	Qui décide
`data.action = list`	non	le code renvoie toute la garde-robe
`data.action = search`	non	le code applique les filtres exacts reçus
`data.action = ask`	**oui**, lecture seule	le LLM choisit les filtres ; le code filtre
texte libre (page Wardrobe, orchestrateur)	**oui**	le LLM choisit les outils ; le code exécute
`check-duplicate`	**oui**	le LLM juge « même pièce vue deux fois ? » ; sans clé OpenAI : « pas de doublon »
recherche : filtres, comparaison, 20 résultats max	non	code
wardrobe_stats, find_similar (score)	non	code
validation d'une correction (_validate_patch)	non	code
confirmation oui / non (regex FR et EN)	non	code
écriture en base (_apply_patch)	non	code

Sans clé OpenAI : les actions list, search et check-duplicate répondent quand même (check-duplicate dit « pas de doublon ») ; le texte libre répond « needs an OPENAI_API_KEY » et ask renvoie une réponse vide.

3. Les 4 outils du LLM

Outil	Paramètres	Effet
`search_wardrobe`	category, category_group, color_name, season, occasion, style, pattern, min_warmth (1 à 5), text	filtre ET ; 20 résultats maximum ; valeurs prises dans des listes fermées (taxonomy.py)
`find_similar`	item_id, ou category, color_name, material, style	pièces proches, score : même type +3, même couleur +2, matière +1, style +1 ; seuil 3 ; 6 meilleures
`wardrobe_stats`	aucun	total, par groupe, catégorie, famille de couleur, saison, occasion, à vérifier, sans photo
`update_item`	item_id, fields	**propose** une correction ; rien n'est sauvegardé

Champs modifiables par l'agent : `name`, `category`, `color_name`, `pattern`, `material`, `style`, `seasons`, `occasions`, `formality`, `warmth`. Les autres sont retirés avant validation.

4. Le flux d'une correction

```
1. Toi : « Change la couleur de ma jupe grise en bleu marine ».
2. gate : aucune proposition en attente, donc assist. Le LLM cherche la jupe, puis appelle
update_item.
3. Le code vérifie la pièce et le champ, la mémorise dans PendingStore (15 minutes). Rien n'est
sauvegardé.
4. Réponse : « Je propose : couleur gris -> bleu marine. Tu confirmes ? » (tâche input-required).
5. Toi : « oui » (ou confirm=true) : gate envoie vers apply, qui écrit par le chemin validé.
6. Toute autre phrase supprime la proposition et devient une nouvelle demande ; « non » l'annule.
```

5. Les prompts

Prompt système principal (`SYSTEM`, wardrobe_agent.py) :

```
You are the Wardrobe Agent of DressMe: you look after the user's wardrobe. Reply in the language the user
wrote in, in one to three plain sentences, no markdown, no emojis. Never invent a piece: always call
search_wardrobe (or find_similar / wardrobe_stats) before talking about what the user owns, and call
search_wardrobe again with other filters when a search comes back empty or too broad. To correct a piece,
find it first, then call update_item: it only PROPOSES the change, so tell the user precisely what would
change (piece, field, old value, new value) and ask them to confirm. Never say a change was made. You do
not give styling advice or build outfits: say that the AI Stylist and the Outfit Builder do that.
```

Ajout quand un autre agent l'appelle (ask, lecture seule) :

```
This request comes from another agent: it is read-only, never propose a change, and answer in one short
sentence listing what you found. Be generous: when the request is about what to wear for an occasion, a
season or the weather, search each role separately (tops, bottoms, dresses, outerwear, shoes) and use
occasion or season filters only when they still return pieces, otherwise search without them: the other
agent chooses among your results. (L'outil update_item est aussi retiré de la liste des outils.)
```

Prompt du contrôle de doublon (`DUPLICATE_SYSTEM`) :

```
You help catalogue a wardrobe that's being scanned live from a camera. An item was just detected and saved.
Decide whether the wardrobe already contains a near-duplicate of it - call search_wardrobe (at least once)
to check, looking at category, colour and whatever else you're told. Only call it a duplicate if you're
genuinely confident it's the same physical garment seen twice (e.g. the user scanned it by accident, or
it's a second photo of the same jacket) - NOT just 'another navy t-shirt' or 'another pair of jeans', which
are normal to own more than one of. When you're done, reply with ONLY a compact JSON object, no other text:
{"duplicate": true or false, "similar_item_id": "<id or null>", "note": "<one short sentence for the user
if duplicate, else empty>"}.
```

Les descriptions des 4 outils (texte vu par le LLM) sont dans les constantes `SEARCH_TOOL`, `FIND_SIMILAR_TOOL`, `STATS_TOOL` et `UPDATE_TOOL` du même fichier. Exemple : update_item dit « PROPOSE a correction ... Nothing is saved: the user must confirm in their next message ».

6. Les paramètres et où les changer

Paramètre	Valeur	Où
Modèle LLM	gpt-4o (variable `LLM_MODEL`)	backend/app/config.py ; surchargé par le .env de la racine
Clé API	`OPENAI_API_KEY` ou `LLM_API_KEY`	backend/app/config.py (lue dans le .env)
Température, réponses	**0,2**	wardrobe_agent.py, `node_assist` (create(... temperature=0.2 ...))
Tokens de sortie, réponses	700 (`max_completion_tokens`)	wardrobe_agent.py, `node_assist`
Température, doublon	**0,1**	wardrobe_agent.py, `check_duplicate`
Tokens de sortie, doublon	250	wardrobe_agent.py, `check_duplicate`
Format de sortie, doublon	JSON strict (`response_format=json_object`)	wardrobe_agent.py, `check_duplicate`
Tours d'outils maximum	6 (réponses) ; 4 (doublon)	`MAX_ROUNDS` et l'argument `max_rounds`
Résultats par recherche	20	`MAX_RESULTS_PER_CALL` dans wardrobe_agent.py
Durée d'une proposition	15 minutes	`PENDING_TTL` dans wardrobe_agent.py
Délai du LLM, essais	45 s, 2 relances	`LLM_TIMEOUT` (config.py) et `max_retries=2`
Longueur d'un message	1 000 caractères	wardrobe_agent.py (`text[:1000]`) et main.py
Délai A2A	30 s (list) ; 60 s (ask)	backend/app/wardrobe_a2a.py
Champs modifiables	10 champs	`EDITABLE` dans wardrobe_agent.py
Mots du oui et du non	regex FR et EN	`YES` et `NO` dans wardrobe_agent.py
Valeurs autorisées (catégories, couleurs...)	listes fermées	backend/app/taxonomy.py

Pour changer la température ou le nombre de tours, il faut éditer wardrobe_agent.py : ces deux valeurs ne sont pas dans le .env. Le modèle et la clé se changent dans le .env.

7. Les fichiers liés

Fichier	Rôle pour le Wardrobe Agent
backend/app/wardrobe_agent.py	l'agent : outils, prompts, graphe, PendingStore, check_duplicate
backend/app/wardrobe_a2a.py	client A2A que les autres agents utilisent pour lui parler (list, ask), avec repli en direct
backend/app/a2a_agents.py	serveur A2A : handle_wardrobe, carte d'agent et 6 compétences
backend/app/main.py	routes /api/wardrobe/agent et /api/agent/check-duplicate ; _validate_patch, _apply_patch ; wardrobe_agent.bind(...)
backend/app/taxonomy.py	listes fermées : catégories, groupes, couleurs, saisons, occasions, styles, motifs
backend/app/config.py	modèle, clé, délai du LLM
backend/app/storage.py	accès Supabase (lecture et écriture des pièces)
agent_kit/tool_loop.py	boucle d'outils LangGraph partagée par tous les agents
a2a_protocol/server.py, client.py	protocole A2A (carte, message/send, tâches)
DressMe UI Blueprint/src/lib/api.ts	wardrobeAgent() et checkDuplicate() côté page
DressMe UI Blueprint/src/features/wardrobe/WardrobePage.tsx	zone de recherche IA, boutons Confirm / Cancel
backend/tests/test_wardrobe_agent.py, test_wardrobe_a2a.py	10 + 6 tests

8. Interface A2A (/a2a/wardrobe)

data.action	LLM ?	Résultat
`list`	non	text « N piece(s) », data.items = toute la garde-robe
`search`	non	data.items trimmés selon data.filters
`ask`	oui, lecture seule	data.reply, data.items, data.filters
`check-duplicate`	oui	data = {duplicate, similar_item_id, note}
(texte libre)	oui	state completed ou input-required ; data.pending si une correction attend

Compétences publiées sur sa carte : wardrobe-search, wardrobe-read, wardrobe-correct, wardrobe-stats, wardrobe-similar, duplicate-check. Le jeton Supabase est exigé ; l'agent ne voit que les pièces de l'appelant.

9. Garde-fous

- **Pas d'outil d'écriture pour le LLM.** update_item ne fait que proposer.
- **Même chemin d'écriture que la page** : _validate_patch (listes fermées, entiers de 1 à 5, nom non vide) puis _apply_patch ; chaque correction est aussi enregistrée pour mesurer la précision du modèle de détection.
- **Isolation** : une proposition est liée à (utilisateur, conversation) ; un autre utilisateur ou une autre conversation ne peut pas la confirmer.
- **Réponses honnêtes** : si le LLM échoue, « The wardrobe agent is having trouble right now » ; si 6 tours ne suffisent pas, « That took more digging than usual » ; si l'écriture est refusée, la raison est donnée.
- **Jamais d'invention** : le prompt impose search_wardrobe avant de parler de ce que l'utilisateur possède, et seules les pièces réellement renvoyées par les outils sont affichées.

10. Limites

- Les filtres sont exacts : « bleu marine » ne trouve pas « bleu ».
- Les propositions et les tâches A2A sont en mémoire du processus : un redémarrage de BO1 les efface.
- Le texte libre dépend d'OpenAI ; sans clé, seules les actions structurées (list, search) fonctionnent.
- Les autres agents lisent toute la garde-robe à chaque demande : correct pour quelques centaines de pièces, à surveiller au-delà.
- L'agent suit les étiquettes de la garde-robe (catégorie, couleur...) : si une pièce est mal classée, il se trompe avec elle.

11. Tests (16, tous verts)

Affirmation	Test
Une correction n'est que proposée	a_correction_is_only_proposed_and_nothing_is_saved_before_the_user_confirms
Écriture seulement après un oui dans la même conversation	the_change_is_written_only_after_a_yes_in_the_same_conversation
Un non, une autre demande ou un autre utilisateur n'écrit jamais	no_or_a_different_request_or_another_user_never_writes
Pièce inconnue, champ interdit, valeur invalide refusés	an_unknown_piece_a_forbidden_field_or_an_invalid_value_is_refused_to_the_llm
Recherche : pièces et filtres renvoyés	a_search_request_returns_the_pieces_the_llm_found_and_the_filters_it_chose
Statistiques et similarité calculées par le code	stats_and_similar_pieces_are_computed_by_code
Contrôle de doublon : outil de recherche et JSON	the_duplicate_check_of_the_live_scan_uses_the_search_tool_and_answers_in_json
Demande vide et LLM en panne	an_empty_request_asks_what_to_do_and_a_failing_llm_is_reported
Boucle d'outils : tours, rappel, abandon propre	the_tool_loop_runs_tools_nudges_once_and_reports_how_it_ended, ..._gives_up_cleanly
A2A : lecture avec le jeton, repli en direct, ask	6 tests de test_wardrobe_a2a.py